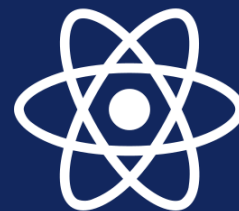


SE3090 – Software Engineering Frameworks

Lecture 02

Advanced React Frontend Development



Year 3 • Semester 1 • 4 Credits

Lecturer: Eishan Weerasinghe | eishan.w@sliit.lk | Course web: courseweb.sliit.lk

Where This Lecture Fits in SE3090

L01

Framework fundamentals & trends

TODAY

Advanced React frontend

L03

C#, .NET & REST API backend

L04

Database, AuthN/AuthZ & integration

Later

Agentic dev & DevOps

- **Module content area: modern frontend frameworks, architecture and production practices**
- Builds on React basics from prior modules — today is the advanced, real-world layer
- The frontend you design today will call the REST APIs you build in Lecture 03 and 04
- Directly assessed in labs, the mini hackathon and the Main Assignment's frontend (UI/UX + code quality)

Lecture Learning Outcomes

LO1

Understand modern framework concepts

Explain component-based architecture, the React ecosystem and React's place in full-stack development.

LO2

Apply suitable frameworks & tools

Build production-grade React: feature structure, reusable components, custom hooks, forms, routing and data fetching.

LO3

Use best practices for quality

Apply separation of concerns, validation, server-state management, protected routes and performance optimization.

LO4

Select & justify approaches

Choose between Context, Redux Toolkit, Zustand, TanStack Query and RTK Query based on requirements.

Why Advanced React Development Matters



Basics don't scale

useState + fetch in every component works for a demo. At 50 components and 5 developers it becomes unmaintainable spaghetti.



Server state is different

Most app data lives on a server. Treating it like local state causes stale data, race conditions and duplicate requests.



Users feel performance

Unnecessary re-renders, huge bundles and unoptimized lists make apps janky. Real users abandon slow pages.



Real apps need structure & security

Roles, protected routes, validation and clean architecture are expected in industry — and graded in your assignment.

Lecture Roadmap – 3 Hours

1

Modern React & production architecture

~30 min

Components · ecosystem · feature-based structure · separation of concerns

2

Component patterns & custom hooks

~40 min

Composition · compound · controlled · headless · useFetch/useDebounce/useAuth ...

3

Advanced forms & validation

~30 min

React Hook Form · Zod/Yup · dynamic fields · error handling

4

API integration, server state & routing

~45 min

Axios · loading/error · TanStack & RTK Query · nested/protected/lazy routes

5

Performance & state-management decisions

~35 min

Memoization · code splitting · virtualization · choosing the right tool

PART 1

Modern React & Production-Level Architecture

- Role of React & component-based architecture
- React in full-stack development & the ecosystem
- Feature-based folder structure
- Separation of UI, logic, services and hooks

The Role of React in Modern Frontend Development

- A JavaScript library for building user interfaces from components
 - Declarative: describe WHAT the UI should look like for a given state
 - React keeps the DOM in sync with your data via a virtual DOM diff
- Component-based: build small pieces, compose them into screens
- Unidirectional data flow: state flows down, events flow up
- Huge ecosystem: routing, data, forms, UI kits, meta-frameworks (Next.js)
- Backed by Meta + a massive community — a safe industry bet

Declarative in one line

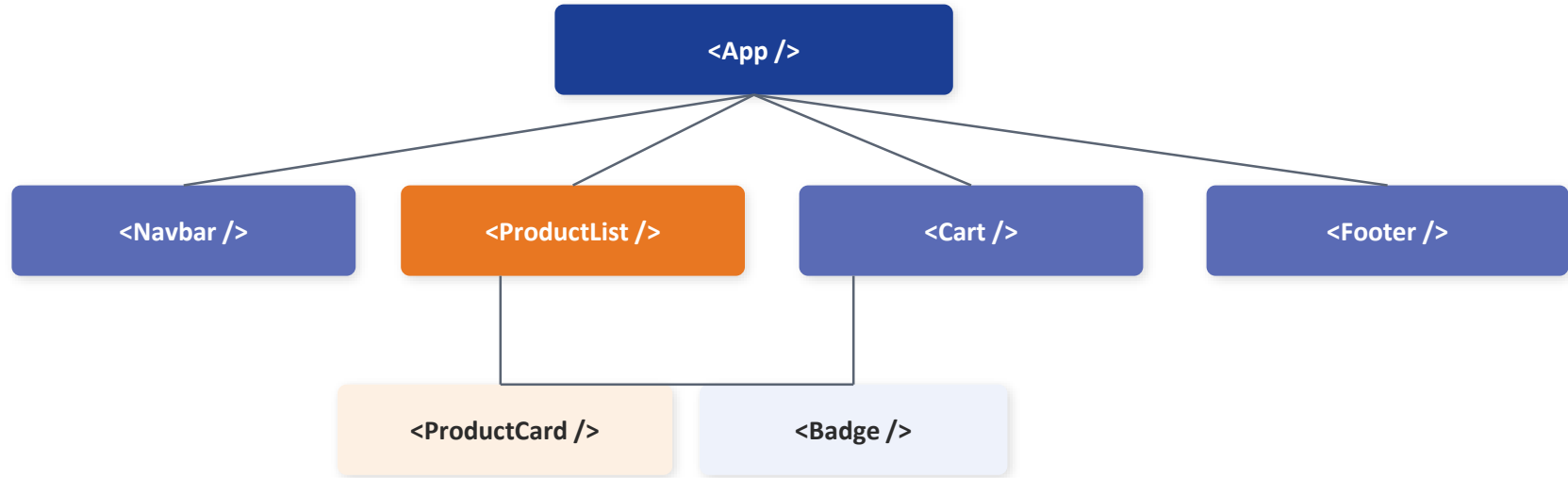
UI = f(state)

Change the state → React re-renders the right parts.
You never touch the DOM by hand.

a component

```
function Greeting({ name }) {  
  return <h1>Hi {name}!</h1>;  
}  
<Greeting name="Amal" />
```

Component-Based Architecture



- **Break a UI into a tree of independent, reusable components**
 - Each component owns its markup, styling and local behaviour
 - Compose small components (Badge, Card) into bigger ones (List, Page)
 - Reuse the same component with different props across the app

React in Full-Stack Application Development



- **React is the CLIENT tier — it renders UI and talks to the backend over HTTP**
- It sends requests (GET/POST/...) and receives JSON, then updates the UI
- It never touches the database directly — always through the API
- Today: the frontend half. Lectures 03–04: the API and database it consumes

The React Ecosystem & Industry Usage



Routing

React Router · TanStack Router · Next.js App Router



Server state

TanStack Query · RTK Query · SWR



Client state

Context · Redux Toolkit · Zustand · Jotai



Forms

React Hook Form · Formik + Zod / Yup



UI components

MUI · shadcn/ui · Chakra · Ant Design



Meta-frameworks

Next.js · Remix — SSR, routing, fullstack

Production Architecture: Feature-Based Structure

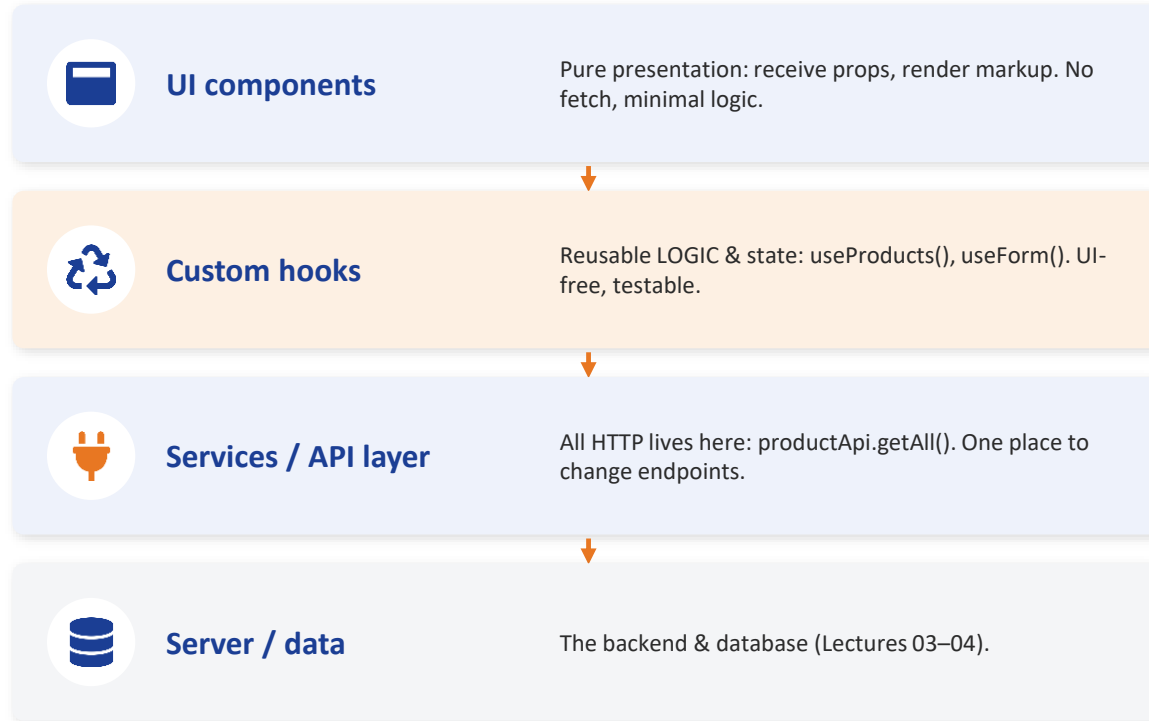
feature-based src/

```
src/
├── components/           // shared, reusable UI
│   ├── Button.jsx
│   ├── Modal.jsx
│   └── Table.jsx
├── layouts/             // page shells
│   └── DashboardLayout.jsx
├── features/            // group by DOMAIN
│   └── products/
│       ├── components/  ProductCard.jsx
│       ├── hooks/       useProducts.js
│       ├── services/    productApi.js
│       └── pages/       ProductListPage.jsx
│   └── auth/ cart/ orders/ ...
├── hooks/              // global custom hooks
├── lib/ services/      // api client, utils
└── routes/             // route config
```

Feature > file-type

- Group by DOMAIN (products, auth), not by type (all components together)
- Everything for a feature lives in one folder — easy to find & delete
- Truly shared UI → top-level components/
- Scales to large teams: features stay independent
- Avoid the 'giant components/ + giant pages/' anti-pattern

Separation of UI, Logic, Services & Hooks



Why separate?

- Test logic without rendering UI
- Swap the API client without touching components
- Reuse a hook across many components
- Components stay small & readable
- Each layer: one reason to change

A Vocabulary of Components



Page components

One per route; compose layout + features. Own data fetching & orchestration.

ProductsPage, CheckoutPage



Layout components

Shared shell: header, sidebar, footer. Wrap pages via layout routes.

DashboardLayout, AuthLayout



Feature components

A slice of business UI; may hold local state and call hooks.

ProductCard, CartSummary



Reusable UI primitives

Generic, domain-free building blocks. The design-system layer.

Button, Modal, Table, Input

Rule of thumb: the lower the layer, the more reusable and the less it should know about your domain.

Checkpoint 01 – Quick Questions



Answer with your neighbour — 4 minutes

Q1

In a feature-based structure, where do the files for the 'orders' feature live — and what goes in its folder?

Q2

A Button component imports the Order model and formats currency. What's wrong with that, and how would you fix it?

Q3

Which layer should contain an axios call: the component, a custom hook, or a services/api module? Why?

Q4

Name one concrete benefit of moving data-fetching logic out of a component and into a custom hook.

PART 2

Component Patterns & Custom Hooks

- Composition, compound components & controlled inputs
- Headless UI and reusable component libraries
- Custom hooks: useFetch, useDebounce, useAuth, usePagination ...
- Separating logic from UI; hook design best practices

Composition over Configuration

```
// ✗ prop explosion - rigid
<Card title="..." subtitle="..." image="..."
      footerButton="Buy" badge="New" .../>

// ✔ composition with children & slots
<Card>
  <Card.Header>{title}</Card.Header>
  <Card.Body><ProductInfo p={p}/></Card.Body>
  <Card.Footer>
    <Button onClick={buy}>Buy</Button>
  </Card.Footer>
</Card>
```

Why compose?

- **children** is React's built-in slot
- Callers control content, not a rigid prop list
- Avoids 15-prop 'god components'
- Each part stays small and replaceable
- Foundation for compound components (next)

Heuristic: if a component keeps gaining boolean/string props to cover new cases, it wants composition instead.

Compound Components — Shared Implicit State

```
// parts share state via context, set by <Tabs>
<Tabs defaultValue="details">
  <Tabs.List>
    <Tabs.Trigger value="details">Details</Tabs.Trigger>
    <Tabs.Trigger value="reviews">Reviews</Tabs.Trigger>
  </Tabs.List>
  <Tabs.Panel value="details">...</Tabs.Panel>
  <Tabs.Panel value="reviews">...</Tabs.Panel>
</Tabs>

// internally: const Ctx = createContext();
// Tabs provides {active,setActive}; parts consume it
```

The idea

- Parent holds state in React Context
- Children read it implicitly — no prop drilling
- Clean, expressive API for the caller
- Flexible markup & ordering
- Pattern behind Tabs, Accordion, Select, Menu

Composition gives structure; compound components add shared state behind the scenes — so `<Tabs.Trigger>` just works.

Controlled vs Uncontrolled Components



Controlled

React state is the single source of truth

- value + onChange bound to state
- Instant validation, formatting, conditional UI
- More re-renders; more wiring
- Use for: live validation, dependent fields



Uncontrolled

The DOM holds the value; read via ref

- defaultValue + ref; read on submit
- Fewer re-renders; less code
- Harder to react to every keystroke
- Use for: simple/large forms, file inputs

```
// controlled
const [q,setQ]=useState("");
<input value={q}
  onChange={e=>setQ(e.target.value)} />
// uncontrolled
const ref=useRef();
<input defaultValue="" ref={ref}/>
```

Headless UI & a Reusable Component Library



Headless UI

- **Logic & accessibility, ZERO styling**
- You own the look; the library owns behaviour (focus, keyboard, ARIA)
- e.g. Radix UI, React Aria, Headless UI, TanStack Table
- Style with Tailwind/CSS to match your brand
- Accessible by default — a huge win

Your shared component library



Modal



Table



Card



Form fields



Inputs



Buttons

Build once, reuse everywhere

- A consistent Modal/Table/Card/Form set enforces design consistency and slashes duplication
- Compose headless behaviour + your styles → accessible, on-brand, maintainable primitives

Custom Hooks — Logic You Can Reuse

- **A custom hook is a function starting with 'use' that calls other hooks**
 - Extracts stateful logic OUT of components so it can be reused & tested
- **Components become 'dumb and pretty'; hooks hold the brains**
- **Same logic, many components — write once, use everywhere**
- **Composable: hooks can call other hooks**
- **The idiomatic React way to share non-visual logic (replaces old HOC/render-props pain)**

```
// before: logic trapped in the component
function ProductList() {
  const [data, setData]=useState();
  const [loading, setLoading]=useState(true);
  useEffect(()=>{ /* fetch... */ }, []);
  ...
}

// after: logic in a reusable hook
function ProductList() {
  const {data, isLoading, error} =
    useFetch('/api/products');
}
```

Essential Custom Hooks (1): Data & Input



`useFetch(url)`

Wraps fetch/axios with { data, isLoading, error }. The bread-and-butter data hook.



`useDebounce(value, ms)`

Delays a fast-changing value (e.g. search text) so you don't fire an API call on every keystroke.



`useLocalStorage(key, init)`

useState that persists to localStorage — survives refresh. Great for theme, drafts, filters.

`useDebounce.js`

```
// useDebounce — the classic
function useDebounce(value, delay=400){
  const [v,setV]=useState(value);
  useEffect(()=>{
    const id=setTimeout(
      ()=>setV(value), delay);
    return ()=>clearTimeout(id);
  },[value,delay]);
  return v; // debounced value
}
```

Essential Custom Hooks (2): App Concerns



`useAuth()`

Exposes the current user, login/logout and isAuthenticated from an Auth context. Used by route guards.



`usePagination(items)`

Holds page, pageSize and helpers (next/prev); returns the current slice. Reused on every list.



`useForm()`

Manages values, errors, touched & submit handling — the core idea behind React Hook Form.

`useAuth.js`

```
// useAuth - consume an Auth context
const AuthCtx = createContext();

export function useAuth(){
  const ctx = useContext(AuthCtx);
  if(!ctx) throw new Error(
    'useAuth outside <AuthProvider>');
  return ctx; // {user,login,logout}
}
```

Custom Hook Design — Best Practices



Name & rules

- Always prefix with use*
- Call hooks at the top level only — never in conditions/loops
- One hook, one responsibility



Clear contract

- Return an object {data,isLoading,error} (named) or a tuple for simple pairs
- Stable function identities (useCallback)
- Document inputs & outputs



Side-effect hygiene

- Always clean up timers, subscriptions, listeners
- Correct dependency arrays
- Avoid stale closures



Composability

- Build big hooks from small ones
- Keep them UI-free (no JSX)
- Make them testable in isolation

Checkpoint 02 – Discussion



Discuss in pairs — 5 minutes, then we compare

D1

Your Card component has grown to 14 props. Which pattern fixes it, and what does the refactored usage look like?

D2

A teammate hand-rolls a `<Dropdown>` with focus-trapping and arrow-key support. Argue for using a headless library instead.

D3

A search box calls the API on every keystroke and the server is overloaded. Which two hooks solve this together, and how?

D4

Where should the user's auth state live, and how does a component read it without prop drilling?

PART 3

Advanced Forms & Validation

- Why complex forms are hard in plain React
- React Hook Form — performant, minimal re-renders
- Schema validation with Zod / Yup
- Dynamic fields, error handling & a real example

Why Complex Forms Are Hard

- **Controlled-everything forms re-render on every keystroke**
 - 30 fields × live state = sluggish, lots of boilerplate
- **Validation is more than 'required'**
 - Cross-field rules (password = confirm), async checks (email taken), formats
- **Error & touched state must be tracked per field**
- **Dynamic fields: add/remove rows (order items, phone numbers)**
- **Submission: disable while sending, show server errors, reset**

```
// the boilerplate trap (per field!)
const [email, setEmail]=useState('');
const [emailErr, setEmailErr]=useState('');
const [touched, setTouched]=useState(false);
// ...repeat ×30 fields...

function validate(){ /* big if-else */ }
// hard to read, easy to get wrong,
// re-renders the whole form constantly
```

This is exactly the pain React Hook Form + a schema validator remove. Next slides →

React Hook Form — Fast, Minimal Re-renders

```
import { useForm } from 'react-hook-form';

const { register, handleSubmit,
  formState: { errors, isSubmitting } }
  = useForm();

<form onSubmit={handleSubmit(onValid)}>
  <input {...register('email')} />
  {errors.email && <span>...</span>}
  <input {...register('password')} />
  <button disabled={isSubmitting}>
    Sign up</button>
</form>
```

Why teams use RHF

- **Uncontrolled under the hood → very few re-renders**
- `register()` wires inputs in one line
- `handleSubmit` runs validation, then your callback
- `formState` gives errors, `isSubmitting`, `isDirty`...
- Tiny, fast, integrates with Zod/Yup

Schema Validation with Zod / Yup

```
import { z } from 'zod';
import { zodResolver } from
  '@hookform/resolvers/zod';

const schema = z.object({
  name: z.string().min(2),
  email: z.string().email(),
  price: z.number().positive(),
  password: z.string().min(8),
  confirm: z.string()
}).refine(d => d.password===d.confirm,
  { message:'Passwords differ',
    path:['confirm'] });

useForm({ resolver: zodResolver(schema) });
```

One schema, many wins

- **Declare rules once, declaratively**
- Cross-field rules via `.refine()`
- Clear, per-field error messages
- Catches bad input at runtime, before use
- Reusable: validate the same shape elsewhere
- Yup is the older sibling — same idea

Dynamic Fields & Error Handling

```
// add/remove rows with useFieldArray
const { fields, append, remove } =
  useFieldArray({ control, name:'items' });

{fields.map((f,i)=>(
  <div key={f.id}>
    <input {...register(`items.${i}.qty`)} />
    <button onClick={()=>remove(i)}>x</button>
  </div>))}
<button onClick={()=>append({qty:1})}>
  + Add item</button>
```

Good error UX

- Show errors after blur/submit, not first keystroke
- Message next to the field, in red, specific
- Summarise on submit; focus the first error
- Map server errors back to fields
- Keep the submit button's state honest

Two error sources: client (schema → instant, friendly) and server (e.g. 409 'email already registered' → map onto the email field). Handle both gracefully.

Putting It Together: A Registration Form



1. Define schema

Zod object with field rules + `.refine()` for password match



2. Wire RHF

`useForm({ resolver: zodResolver(schema) });` register inputs



3. Validate

`handleSubmit` runs the schema; errors shown inline per field



4. Submit

POST to `/api/auth/register`; disable button while sending



5. Handle result

Success → redirect/login; 409 → map server error to email field

Reusable takeaways

- This pattern fits ANY form: login, checkout, settings
- Schema + RHF + good errors = robust & maintainable
- Client validates for UX; server validates for safety
- Reuse the schema to validate on both client and API

PART 4

API Integration, Server State & Routing

- Axios / Fetch best practices · loading, success & error states
- Client state vs server state — the key distinction
- TanStack Query & RTK Query
- Nested, protected, role-based and lazy routes

Axios / Fetch Best Practices & Request States

```
// one configured axios instance (services/api.js)
export const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL });

// interceptor attaches the auth token
api.interceptors.request.use(cfg => {
  cfg.headers.Authorization =
    `Bearer ${getToken()}`;
  return cfg; });

// callers just use endpoints
export const getProducts = () =>
  api.get('/products').then(r => r.data);
```

Best practices

- One configured instance; baseURL from env
- Interceptors for auth & error handling
- Centralise endpoints in a services layer
- Always handle errors — no silent failures

Loading

spinner / skeleton

Success

render the data

Error

message + retry

Every data fetch has three states — design the UI for all three, every time.

Client State vs Server State — The Key Distinction



Client state

Owned by the UI; you control it fully

- Modal open/closed, active tab, theme
- Form input values, wizard step
- Selected filters before applying
- **Tools: useState, Context, Redux, Zustand**



Server state

Lives on the server; you only borrow a copy

- Products, orders, user profile from the API
- Can be stale; others may change it
- Needs caching, refetch, sync, dedupe
- **Tools: TanStack Query, RTK Query**

Why it matters: storing API data in Redux/useState forces you to hand-build caching, loading, refetching and invalidation — server-state libraries give all of that for free.

TanStack Query — Server State Done Right

```
// read: caching, loading & error built in
const { data, isLoading, error } = useQuery({
  queryKey: ['products', { category }],
  queryFn: () => getProducts(category),
});

// write: mutate + invalidate to refetch
const qc = useQueryClient();
const add = useMutation({
  mutationFn: createProduct,
  onSuccess: () => qc.invalidateQueries(
    { queryKey: ['products'] }) });
```

Out of the box

- **Caching by queryKey**
- Automatic loading & error states
- Background refetch & stale-while-revalidate
- Request dedupe across components
- Mutations + cache invalidation
- Pagination & infinite scroll helpers

queryKey is the cache identity: same key → shared cached data; change a key part (e.g. category) → it fetches and caches that variant. Invalidate a key to refetch.

RTK Query — When You're Already on Redux Toolkit

```
// define an API slice once
export const api = createApi({
  baseQuery: fetchBaseQuery({ baseUrl: '/api' }),
  endpoints: (b) => ({
    getProducts: b.query({ query: () => 'products' }),
    addProduct: b.mutation({
      query: (body) => ({ url: 'products',
        method: 'POST', body }),
    })
  })
});

// auto-generated hooks!
const { data, isLoading } = useGetProductsQuery();
```

Why & when

- **Built into Redux Toolkit**
- Auto-generates query/mutation hooks
- Caching, dedupe, invalidation — like TanStack
- Tag-based cache invalidation
- **Pick it if Redux is already in the app**
- Else TanStack Query is lighter for pure server state

Pagination, Filtering, Optimistic Updates & Caching



Pagination

Encode page & pageSize in the queryKey; keep previous data while the next page loads for a smooth UI.



Filtering

Filters/search go in the key too — each combination is its own cached query; debounce the search input.



Optimistic updates

Update the cache immediately, then confirm with the server; roll back on error. Feels instant.



Caching & staleness

staleTime controls when data refetches; cached data shows instantly while a fresh copy loads in the background.

Advanced Routing: Nested, Layout & Lazy Routes

```
// React Router - nested + layout + lazy
const Settings = lazy(()=>import('./Settings'));

<Routes>
  <Route element={<DashboardLayout/>}>
    <Route path='products' element={<Products/>}/>
    <Route path='products/:id'
      element={<ProductDetail/>}/>
    <Route path='settings' element={<Settings/>}/>
  </Route>
  <Route path='*' element={<NotFound/>}/>
</Routes>
```

Concepts

- **Layout route renders shared shell + <Outlet/>**
- Nested routes share that layout
- `:id` = URL parameter (useParams)
- `lazy()` + `Suspense` → code-split per route
- `path='*' → 404 / error page`

Layout routes remove repetition (one header/sidebar for many pages); lazy routes keep the initial bundle small.

Protected & Role-Based Routes (Route Guards)

```
// a guard component using useAuth()
function RequireAuth({ roles }) {
  const { user } = useAuth();
  const loc = useLocation();
  if (!user)
    return <Navigate to='/login'
      state={{from: loc}} replace/>;
  if (roles && !roles.includes(user.role))
    return <Navigate to='/forbidden' replace/>;
  return <Outlet/>; // allowed
}

<Route element={<RequireAuth roles={['Admin']}/>}>
  <Route path='admin' element={<AdminPanel/>}/>
</Route>
```

How guards work

- Wrap routes in a guard that checks auth/role
- Not logged in → redirect to /login (remember origin)
- Wrong role → /forbidden
- Allowed → render <Outlet/>
- **UI hiding is UX only — the API must enforce real security**

Error Pages & a Look at the Next.js App Router



Robust error handling

- 404 route (path=('*')) for unknown URLs
- Error boundaries catch render-time crashes
- Friendly message + a way back, not a blank screen
- Per-route loading & error UI
- Log errors for monitoring



Next.js App Router (overview)

- File-system routing: folders = routes
- layout.jsx, page.jsx, loading.jsx, error.jsx
- Server Components & data fetching on the server
- Built-in code splitting & image optimization
- Great for SEO & performance (SSR/SSG)

React Router vs Next.js: React Router = routing for a client-side SPA (what we use here). Next.js = a full framework adding server rendering, file routing and built-in optimizations. Choose per project needs (LO4).

Checkpoint 03 – Discussion



Discuss in pairs — 5 minutes, then we compare

D1

Classify each as client or server state: theme toggle, product list, modal open, logged-in user's orders, current wizard step.

D2

A teammate stores all API data in Redux and writes loading flags by hand. What would you suggest instead, and why?

D3

TanStack Query vs RTK Query — give one concrete situation where each is the better pick.

D4

A user disables JavaScript guards and navigates to `/admin`. Is your data safe? What actually protects it?

PART 5

Performance Optimization & State-Management Decisions

- Profiling & detecting unnecessary re-renders
- React.memo · useMemo · useCallback
- Lazy loading, code splitting & list virtualization
- Choosing the right state tool — the decision table

Profiling & Unnecessary Re-Renders

- **Rule #1: measure before optimizing — guesses are usually wrong**
- **React DevTools Profiler records what rendered, why, and how long**
 - "Highlight updates" visually flashes components as they re-render
- **A component re-renders when its state, props or context change...**
 - ...and ALL its children re-render by default — even if their props didn't change
- **Common culprits:**
 - New object/array/function created inline on every render
 - Context value changing too often; state lifted too high



Profiler workflow

1. Record an interaction
2. Read the flame graph (wide = slow)
3. Check 'why did this render?'
4. Fix the cause → re-measure

```
// ✗ new array every render → child re-renders
<List items={data.filter(x => x.active)} />
// ✔ memoize the derived value
const active = useMemo(
  () => data.filter(x => x.active),
  [data]);
```

React.memo, useMemo & useCallback

Tool	What it memoizes	Use when...
React.memo	A whole component's render output (skips render if props are equal)	An expensive child re-renders with the same props
useMemo	A computed VALUE between renders	A calculation is costly, or you need a stable object/array reference
useCallback	A FUNCTION reference between renders	Passing a callback to a memoized child or a hook dependency

```
const Row = React.memo(({ item, onPick }) => ( ... )); // skips equal-props renders
const total = useMemo(() => heavy(items), [items]); // recompute only if items change
const onPick = useCallback((id) => select(id), []); // stable fn → memo stays effective
```

These have a cost (memory + comparisons). Apply them to **PROVEN** hotspots, not everywhere. **useCallback** is pointless unless the child is memoized.

Bundles, Lazy Loading & List Virtualization



Code splitting + lazy

- Don't ship the whole app up front
- React.lazy + <Suspense> per route
- Smaller initial bundle → faster first load



Route-based splitting

- Each page becomes its own chunk
- Admin code not downloaded by shoppers
- Pairs with lazy routes from Part 5 routing



List virtualization

- Render only visible rows (windowing)
- react-window / TanStack Virtual
- 1000s of rows stay smooth



Image optimization

- Right size & modern formats (WebP)
- Lazy-load below-the-fold images
- Often the biggest payload on a page

State Management — Know Your Options

- **First, classify the state — this decides the tool**
 - Local UI state (is the modal open?) → local useState
 - Shared client state (theme, current user) → Context or a store
 - Server state (data from the API) → a server-cache library
- **Most 'state management pain' is using a CLIENT tool for SERVER data**

The key reframe

Client state and server state are different problems. Server state is shared, async, cached and can go stale — purpose-built libraries (TanStack/RTK Query) handle fetching, caching and refetching so you don't reinvent it in Redux.

The mainstream toolbox

useState Local UI state

Context API Low-frequency shared state

Redux Toolkit Large, complex client state

Zustand Simple global store

TanStack Query Server state (any backend)

RTK Query Server state (Redux apps)

State Management Decision Table (LO4)

If you need...	Use	Why / notes
State used by one component	<code>useState</code>	Simplest; never reach for more until you must
A few components share rarely-changing data	<code>Context API</code>	Theme, locale, current user; avoid for fast-changing values
Large, complex, interconnected client state	<code>Redux Toolkit</code>	Predictable, debuggable, DevTools; more setup
Simple global state, minimal boilerplate	<code>Zustand</code>	Tiny API, no providers; great middle ground
Data from an API (any stack)	<code>TanStack Query</code>	Caching, loading/error, refetch, pagination built-in
API data + already using Redux	<code>RTK Query</code>	Same benefits, integrated with the Redux store

Golden rule: start local, lift only when sharing is needed, and keep server state OUT of client stores. Justify every choice against the requirement — that is LO4.

Case Study: Designing the LankaMart Frontend



Requirements → frontend design decisions (with justification — LO4)

Requirement	Design decision	Justification
Organise a growing codebase	Feature-based structure: features/products, cart, auth	Locality & team scaling; easy to find and delete features
Browse & search products	TanStack Query + useDebounce on the search box	Server data cached; debounce avoids a request per keystroke
Reusable UI everywhere	Shared Button, Modal, Table; compound <Table> pattern	Consistency; build once, use across features
Checkout form with validation	React Hook Form + Zod schema	Performant, declarative validation; one shared schema
Admin-only dashboard	Protected + role-based, lazy-loaded route	Security gate (server-enforced) + smaller shopper bundle
Fast product listing	List virtualization + image optimization	Smooth scrolling for large catalogues
Cart state while shopping	Zustand (small global store)	Simple global client state; no async, no Redux overhead

Group Activity: Choose the Architecture (20 min)



Groups of 3–4 · whiteboard or paper · present in 60 seconds

Scenario: Campus Event Portal

Students browse & search events, register (form), and see 'my tickets'. Organisers have an admin area to create events and view attendees.

- **Task 1 — Sketch the feature-based folder structure**
- **Task 2 — For each piece of state (search results, the registration form, the theme, the logged-in user, the cart of tickets), pick a tool & justify it**
- **Task 3 — Name two reusable components and one custom hook you'd build**
- **Task 4 — Which routes are protected / role-based / lazy-loaded?**

Marking lens (= the rubric)

- Feature-based, not type-based structure?
- Server state via a query library, not Redux/Context?
- Local state kept local; right tool per state?
- Reusable components identified, logic in hooks?
- Admin routes protected (and server-enforced) + lazy?

Quick Quiz — 6 Questions

1

Data fetched from your REST API is which kind of state?

A) local UI B) server state C) form state

2

Best default tool for caching API data with loading/error built in?

A) Redux B) Context C) TanStack Query

3

React.memo on a child still re-renders. Most likely cause?

A) React bug B) an unstable prop C) too much CSS

4

Which hook gives a STABLE function reference?

A) useMemo B) useEffect C) useCallback

5

A frontend route guard hides /admin. Is the data secure?

A) yes B) only if the API enforces authz C) yes, if lazy

6

200 product rows scroll slowly. Best fix?

A) React.memo all B) list virtualization C) remove images

Lecture Summary — Key Takeaways



Architect for growth

Feature-based structure; separate UI, logic (hooks) & services.



Treat server state specially

TanStack/RTK Query for caching, loading, errors & refetch — not Redux.



Design reusable components

Composition, compound components, controlled inputs & headless UI.



Route & optimize deliberately

Protected & lazy routes; profile, memoize, virtualize, split bundles.



Extract logic into hooks

useFetch, useDebounce, useAuth... thin components, smart hooks.



Choose tools by requirement

Local → Context → store; client vs server state. Justify every choice (LO4).

Learning Outcomes, Next Lecture & Preparation

LO	Covered today by...
LO1	Component model, the React ecosystem, and the client-state vs server-state distinction
LO2	Feature-based architecture, reusable components, custom hooks, forms, routing & data fetching
LO3	Performance optimization, validation, error handling and separation of UI/logic/services
LO4	The state-management decision table and the case study / architecture activity



Next: Lecture 03 — C#, .NET & REST APIs

Build the backend this frontend will call: C#/.NET fundamentals and ASP.NET Core REST API development — the GET/POST endpoints behind today's components.



Before the lab / next week

- Scaffold a Vite + React (JavaScript) app; set up the feature-based folders
- Build one reusable component and one custom hook (e.g. useDebounce)
- Read: React docs 'Thinking in React' & TanStack Query overview
- Sketch the component tree for YOUR assignment frontend

References & Further Reading

- **Official documentation (primary sources)**

- React Documentation — react.dev (Learn React, 'Thinking in React', Hooks reference)
- TanStack Query & TanStack Virtual — tanstack.com
- Redux Toolkit & RTK Query — redux-toolkit.js.org
- React Router — reactrouter.com · Zustand — docs.pmnd.rs
- React Hook Form — react-hook-form.com · Zod — zod.dev

- **Guides & tooling**

- Vite — vitejs.dev · React DevTools Profiler (Chrome/Firefox extension)
- web.dev — performance, Core Web Vitals & image optimization (Google)

- **Recommended reading**

- Banks, A. & Porcello, E. (2020) Learning React, 2nd edn. O'Reilly Media.
- Roldán, C. S. (2021) React Design Patterns and Best Practices. Packt.